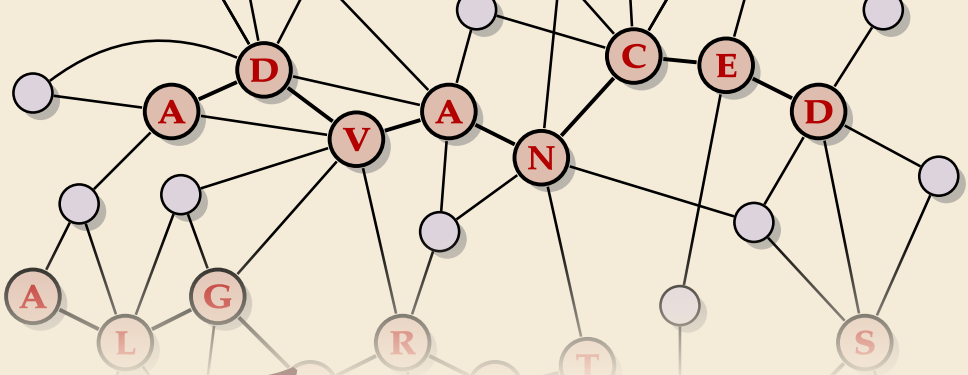


4 Fixed-Parameter Algorithms

Advanced Algorithms · Winter 2026–27

Prof. Dr. Sebastian Wild

CS627 (Winter 2026–27)
Philipps-Universität Marburg
version `2026-10-26 09:14`

Outline

4 Fixed-Parameter Algorithms

4.1 Fixed-Parameter Tractability
4.2 Depth-Bounded Exhaustive Search I
4.3 Problem Kernels
4.4 Depth-Bounded Search II: Planar Independent Set
4.5 Depth-Bounded Search III: Closest String
4.6 Linear Recurrences & Better Vertex Cover
4.7 Interleaving

Philosophy of FPT

- **Goal:** Principled theory for studying complexity based on two dimensions: input size $n = |x|$ (encoding length) and *some additional parameter* k
  - generalize ideas from $k = MaxInt(x)$
  - investigate influence of k (and n) on running time

Philosophy of FPT

- **Goal:** Principled theory for studying complexity based on two dimensions: input size $n = |x|$ (encoding length) and *some additional parameter* k
  - generalize ideas from $k = MaxInt(x)$
  - investigate influence of k (and n) on running time

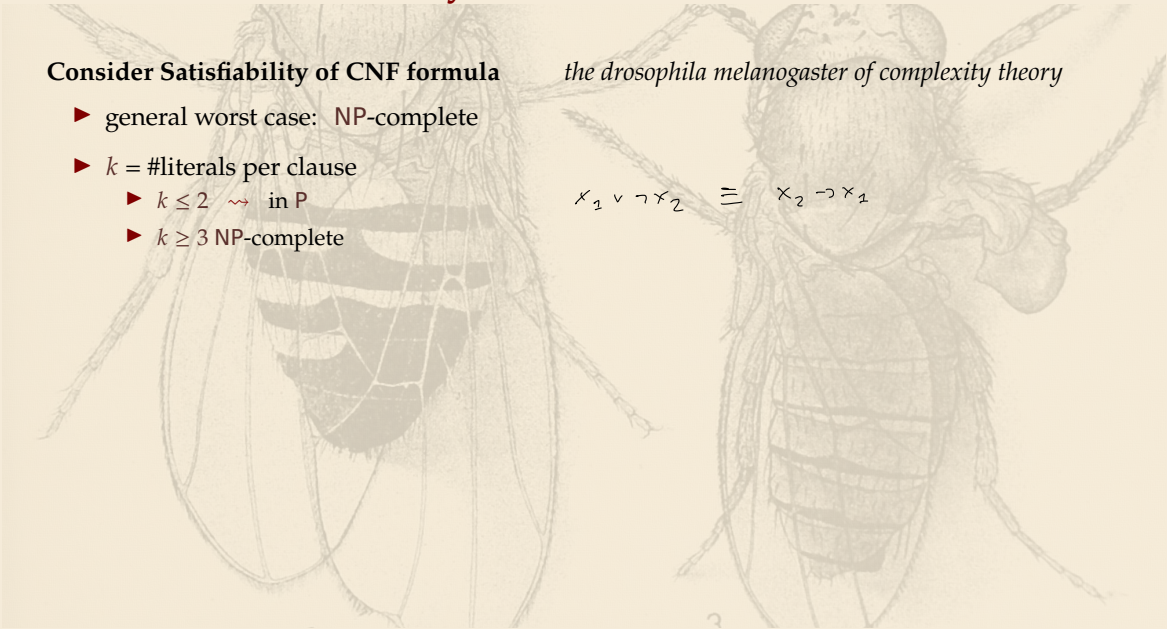
  $\rightsquigarrow$ Try to find a parameter k such that
  (1) the problem can be solved efficiently as long as k is small, and
  (2) practical instances have small values of k (even where n gets big).

Motivation: Satisfiability

Consider Satisfiability of CNF formula *the drosophila melanogaster of complexity theory*

- general worst case: NP-complete
- k = #literals per clause
  - $k \leq 2 \quad \rightsquigarrow$ in P
  - $k \geq 3$ NP-complete

$x_1 \vee \neg x_2 \quad \equiv \quad x_2 \rightarrow x_1$



Motivation: Satisfiability

Consider Satisfiability of CNF formula *the drosophila melanogaster of complexity theory*

- general worst case: NP-complete
- k = #literals per clause
  - $k \leq 2 \rightsquigarrow$ in P
  - $k \geq 3$ NP-complete
- k = #variables
  - $O(2^k \cdot n)$ time possible (try all assignments)
- k = #clauses?
- k = #literals?
- k = #ones in satisfying assignment
- k = structural property of formula
- for MAX-SAT, k = #optimal clauses to satisfy

more reasonable

$\approx n$

Parameters

Definition 4.1 (Parameterization)

Let $\Pi = (L, Y, \Sigma)$ be a decision problem. A *parameterization* of Π is a polytime computable mapping $\kappa : L \to \mathbb{N}$. ◀

Definition 4.2 (Parameterized problem)

A *parameterized (decision) problem* is a pair (Π, κ) of a decision problem Π and a parameterization κ of Π. ◀

Examples

As before: Typically leave encoding implicit.

Definition 4.3 (p-variables-SAT)

Given: formula boolean ϕ (same as before)

Parameter: number of variables

Question: Is there a satisfying assignment $v : [n] \to \{0,1\}$? ◀

Definition 4.4 (p-Clique)

Given: graph $G = (V, E)$ and $k \in \mathbb{N}$

Parameter: k

Question: $\exists V' \subseteq V \;:\; |V'| \geq k \;\wedge\; \forall u, v \in V' : \{u, v\} \in E$? ◀

Definition 4.5 (p-Vertex-Cover)

Given: graph $G = (V, E)$ and $k \in \mathbb{N}$

Parameter: k

Question: $\exists S \subseteq V \;:\; |S| \leq k \;\wedge\; \forall e \in E : S \cap e \neq \emptyset$? ◀

Canonical Parameterization

Typical use case: study (decision versions of) optimization problems

Definition 4.6 (Canonical Parameterization)

If the instances of a decision problem Π are pairs (x, k) with $k \in \mathbb{N}$, the *canonical parameterization* is $\kappa(x, k) = k$. We write $p\text{-}\Pi$ for (Π, κ).

For $\Pi = (\Sigma_I, \Sigma_O, L, L_I, M, value, goal)$ an optimization problem, $p\text{-}\Pi$ denotes the *(canonically) parameterized (decision) problem* given by the threshold problem $Threshold_\Pi$:
$p\text{-}\Pi = p\text{-}Threshold_\Pi$. ◀

Recall: $Threshold_\Pi$ is the set of pairs (x, k) of all instances $x \in L_I$ that have solutions that are weakly "better" than k.

Canonical Parameterization

Typical use case: study (decision versions of) optimization problems

Definition 4.6 (Canonical Parameterization)

If the instances of a decision problem Π are pairs (x, k) with $k \in \mathbb{N}$, the *canonical parameterization* is $\kappa(x, k) = k$. We write $p\text{-}\Pi$ for (Π, κ).

For $\Pi = (\Sigma_I, \Sigma_O, L, L_I, M, \mathit{value}, \mathit{goal})$ an optimization problem, $p\text{-}\Pi$ denotes the *(canonically) parameterized (decision) problem* given by the threshold problem $\mathit{Threshold}_\Pi$:
$p\text{-}\Pi = p\text{-}\mathit{Threshold}_\Pi$. ◀

Recall: $\mathit{Threshold}_\Pi$ is the set of pairs (x, k) of all instances $x \in L_I$ that have solutions that are weakly "better" than k.

Examples:

- p-Clique
- p-Vertex-Cover
- p-Graph-Coloring
- ...

Naming convention for other parameters:
p-clause-CNF-SAT: CNF-SAT with parameter "number of *clauses*"

4.1 Fixed-Parameter Tractability

Exemplary Running Times of Parameterized Problems

- *p-variables*-SAT
  - k variables, n length of formula
  - $\rightsquigarrow$ $O(2^k \cdot n)$ running time

(consider simplest brute-force methods for problems)

Exemplary Running Times of Parameterized Problems

(consider simplest brute-force methods for problems)

- p-*variables*-SAT
  - k variables, n length of formula
  - $\rightsquigarrow$ $O(2^k \cdot n)$ running time
- p-Clique
  - k threshold (clique size); n vertices, m edges in graph
  - $\rightsquigarrow$ $\binom{n}{k}$ candidates to check, each takes time $O(k^2)$ to check
  - $\rightsquigarrow$ Total time $O(n^k \cdot k^2)$

Exemplary Running Times of Parameterized Problems

- p-*variables*-SAT (consider simplest brute-force methods for problems)
  - k variables, n length of formula
  - $\rightsquigarrow$ $O(2^k \cdot n)$ running time
- p-Clique
  - k threshold (clique size); n vertices, m edges in graph
  - $\rightsquigarrow$ $\binom{n}{k}$ candidates to check, each takes time $O(k^2)$ to check
  - $\rightsquigarrow$ Total time $O(n^k \cdot k^2)$
- p-VertexCover
  - k threshold (VC size); n vertices, m edges in graph
  - $\rightsquigarrow$ $\binom{n}{k}$ candidates to check, each takes time $O(m)$ to check
  - $\rightsquigarrow$ Total time $O(n^k \cdot m)$

Exemplary Running Times of Parameterized Problems

- p-*variables*-SAT (consider simplest brute-force methods for problems)
  - k variables, n length of formula
  - $\rightsquigarrow$ $O(2^k \cdot n)$ running time
- p-Clique
  - k threshold (clique size); n vertices, m edges in graph
  - $\rightsquigarrow$ $\binom{n}{k}$ candidates to check, each takes time $O(k^2)$ to check
  - $\rightsquigarrow$ Total time $O(n^k \cdot k^2)$
- p-VertexCover
  - k threshold (VC size); n vertices, m edges in graph
  - $\rightsquigarrow$ $\binom{n}{k}$ candidates to check, each takes time $O(m)$ to check
  - $\rightsquigarrow$ Total time $O(n^k \cdot m)$
- p-GraphColoring
  - k threshold (#colors); n vertices, m edges in graph
  - $\rightsquigarrow$ k^n candidates to check, each takes time $O(m)$
  - $\rightsquigarrow$ Total time $O(k^n \cdot m)$